



Restrições de Integridade e Stored Procedures

Álvaro G. Pagliari
alvaro.pagliari@unoesc.edu.br



- Restrições de Integridade
- Funções
- Exercícios :)



Restrições de Integridade (RIs)

Restrições

- Restrições (constraints) são construções previstas no SQL utilizados para definir um nível de controle mais apurado sobre os dados e seu domínio
- Geralmente, restrições podem ser definidas de duas maneiras
 - Através de DDL na construção de tabelas
 - Unique, not null, check
 - Adicionados posteriormente sob demanda
 - constraint



- Checagem (check)
 - Permitem a especificação de valores válidos ou não para o campo
 - Podem referenciar outros campos
 - Avaliam uma expressão booleana



- Exemplo

```
create table Empregados (  
  
    codEmp integer,  
    nome varchar(40) not null,  
    CPF numeric(11) not null unique,  
    idade integer check (idade between 16 and 90),  
    estadoCivil char(10) check (estadoCivil in  
        ('solteiro', 'casado', 'viúvo', 'desquitado',  
        'divorciado')),  
    salario numeric(8,2) check (salario > 0),  
    tempoServiço integer,  
    codGer integer,  
    codDepto integer,  
    ...
```

Restrições

- Exemplo

```
...  
constraint EmpPk primary key (codEmp),  
constraint EmpCodg foreign key (codGer)  
    references Empregados on update cascade  
                        on delete set null,  
constraint EmpDept foreign key (codDepto)  
    references Departamentos on update cascade  
                        on delete no action,  
constraint IdadeTS check(tempoServico < idade),  
constraint EmpGer check(codEmp < > codGer),  
constraint SalarioGerente  
    check(salario < (select salario  
                    from Empregados e  
                    where e.codEmp = codGer))  
);
```

Restrições

- Exemplo

```
...  
constraint EmpPk primary key (codEmp),  
constraint EmpCodg foreign key (codGer)  
    references Empregados on update cascade  
                        on delete set null,  
constraint EmpDept foreign key (codDepto)  
    references Departamentos on update cascade  
                        on delete no action,  
constraint IdadeTS check(tempoServico < idade),  
constraint EmpGer check(codEmp < > codGer),  
constraint SalarioGerente  
    check(salario < (select salario  
                    from Empregados e  
                    where e.codEmp = codGer))  
  
);
```


Restrições

- Exemplo

```
create table Departamentos (  
  
    codDepto integer,  
    nome varchar(20) check (nome in ('Vendas',  
        'Pessoal', 'Finanças', 'Administrativo')),  
    Andar integer check (andar between 1 and 10),  
    orçamento numeric(20,2),  
  
    constraint DeptPk primary key (codDepto),  
    constraint ControleOrç  
        check ( orçamento > =  
            (select sum(salário)  
                from Empregados e  
                where e.codDepto = codDepto) )  
);
```

Restrições

- Exemplo

```
create table Departamentos (  
  
    codDepto integer,  
    nome varchar(20) check (nome in ('Vendas',  
        'Pessoal', 'Finanças', 'Administrativo')),  
    Andar integer check (andar between 1 and 10),  
    orçamento numeric(20,2),  
  
    constraint DeptPk primary key (codDepto),  
    constraint ControleOrç  
        check( orçamento > =  
            (select sum(salário)  
            from Empregados e  
            where e.codDepto = codDepto))  
  
);
```

- Exemplo (Inclusão posterior)

ALTER TABLE departamentos **ADD CONSTRAINT**
nome_departamento **CHECK** (nome **IN** ('Vendas',
'Pessoal', 'Financas'))

Funções



Funções

- Programação estruturada
 - Procedimentos
 - Funções
- Em BD
 - Geralmente procedures contém rollback e funções não
- Postgres
 - Tudo a mesma coisa



CREATE [OR REPLACE] FUNCTION

name([[argname] argtype])

[**RETURNS** tipo | [**TABLE** (cols)]]

AS \$\$

[**DECLARE** var tipo;]

BEGIN

operações

END;

\$\$ LANGUAGE plpgsql;



Exemplo – condicional

```
CREATE OR REPLACE FUNCTION numero_par (i int)
RETURNS boolean AS $$
DECLARE
    temp int;
BEGIN
    temp := i % 2;
    IF temp = 0 THEN RETURN true;
    ELSE RETURN false;
    END IF;
END;
$$ LANGUAGE plpgsql;

SELECT numero_par(3), numero_par(42);
```



Exemplo – laço FOR

```
CREATE OR REPLACE FUNCTION fatorial (i numeric)
RETURNS numeric AS $$
DECLARE
    temp numeric; resultado numeric;
BEGIN
    resultado := 1;
    FOR temp IN 1 .. i LOOP
        resultado := resultado * temp;
    END LOOP;
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;

SELECT fatorial(42::numeric);
```





Exemplo – laço WHILE

```
CREATE OR REPLACE FUNCTION fatorial (i numeric)
RETURNS numeric AS $$
DECLARE temp numeric; resultado numeric;
BEGIN
    resultado := 1; temp := 1;
    WHILE temp <= i LOOP
        resultado := resultado * temp;
        temp := temp + 1;
    END LOOP;
    RETURN resultado;
END;
$$ LANGUAGE plpgsql;

SELECT fatorial(42::numeric);
```



Exemplo – SQL dinâmico

```
CREATE OR REPLACE FUNCTION recupera_funcionario(id int)
RETURNS funcionario AS $$
DECLARE
    registro RECORD;
BEGIN
    EXECUTE 'SELECT * FROM funcionario WHERE id = ' || id INTO
registro;
    RETURN registro;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM recupera_funcionario(1);
```

Exemplo – cursor

```
CREATE OR REPLACE FUNCTION total_salarios()  
RETURNS numeric AS $$  
DECLARE  
    registro RECORD; resultado numeric;  
BEGIN  
    resultado := 0.00;  
    FOR registro IN SELECT * FROM funcionario LOOP  
        resultado := resultado + registro.salario;  
    END LOOP;  
    RETURN resultado;  
END;  
$$ LANGUAGE plpgsql;  
  
SELECT total_salarios();
```

Exercícios (parte 1)



- 1) Para as seguintes tabelas, crie as seguintes restrições de checagem:
 - a) "Ambulatorios": andar ≥ 0 , capacidade ≥ 1
 - b) "Medicos": idade ≥ 16
 - c) "Pacientes": idade ≥ 0
 - d) "Funcionarios": idade ≥ 0 , salario ≥ 0
- 2) Para as tabelas "Medicos", "Pacientes" e "Funcionarios", crie restrições de unicidade para o campo "CPF" (se não existirem)
- 3) Crie as funções apresentadas nos slides de 15 a 19 em sua instância de banco de dados. Quando necessário, faça adaptações no código para utilizar as suas tabelas no lugar das tabelas utilizadas nos slides.



Exercícios (parte 2)

- 4) Uma prática utilizada durante o desenvolvimento de aplicações que interagem com bancos de dados é a de definir procedimentos ou funções responsáveis pela inclusão, alteração e exclusão de registros. Para as tabelas de Pacientes e Médicos, crie funções que atendam a essas operações, respeitando as seguintes regras:
- a) no caso de inclusões, a função deverá retornar a chave primária do novo registro como resultado;
 - b) no caso de alterações, a chave primária cujo registro deverá ser modificado deverá ser passada como parâmetro (juntamente com os dados a serem modificados no registro). O retorno dessa função deverá ser nulo;
 - c) no caso de exclusões, a chave primária cujo registro deverá ser removido deverá ser passada como parâmetro. O retorno dessa função deverá ser nulo.